Aplicación de Chat en Rust

Jorge Iván Reyes Hernández

27 de marzo de 2026

Resumen

Se describen la arquitectura y las decisiones de diseño tomadas en la implementación de la aplicación de chat. La aplicación sigue una arquitectura cliente-servidor, en la cual se utiliza el patrón MVC para el cliente, comunicación mediante sockets TCP sobre IP, serialización JSON para el protocolo de mensajes, un sistema de broadcast para la distribución de mensajes en tiempo real entre hilos en el servidor.

1. Introducción

El sistema se desarrolló en Rust por recomendación de Canek, y porque Rust es un lenguaje de programación de bajo nivel, sin recolector de basura, sin fugas de datos (como en C). Esto lo vuelve sumamente atractivo para implementar sistemas críticos que deben correr por mucho tiempo sin fallas, como un servidor.

2. Arquitectura del sistema

2.1. Arquitectura Cliente-Servidor

El sistema implementa una arquitectura cliente-servidor donde un servidor central gestiona todas las conexiones de clientes. El servidor escucha en un puerto TCP y acepta conexiones entrantes.

Cuando un cliente se conecta, el servidor crea dos nuevos hilos dedicados para manejar esa conexión específica, uno para lectura y otro para escritura. Este enfoque permite que el servidor maneje múltiples clientes simultáneamente sin bloqueos. El servidor mantiene un estado compartido que incluye:

- Diccionario de usuarios conectados con sus estados. Los nombres de usuario son las llaves, y los valores son una estructura User que encapsula su nombre, estado y id en el servidor.
- Diccionario de los cuartos creados. El nombre del cuarto es la llave y su respectivo valor es una estructura Room que encapsula a los usuarios dentro de ese cuarto y la lista de invitados al mismo.

2.2. Patrón MVC en el Cliente

El cliente implementa el patrón Modelo-Vista-Controlador para separar responsabilidades:

Modelo: Almacena el estado local de la aplicación, incluyendo información del usuario local, lista de usuarios remotos, salas de chat, estado de conexión y mensajes recibidos.

Vista: Implementa la interfaz de usuario. Se definió un trait de Rust con el comportamiento que debe tener cualquier vista para la aplicación. En particular, se implementó una vista en linea de comandos, dada por `ConsoleView`. La vista es responsable de mostrar mensajes, listas de usuarios, notificaciones y errores. Utiliza entrada/salida estándar para interactuar con el usuario.

Controlador: Actúa como intermediario entre vista y modelo. Procesa los eventos realizados por el usuario (detectados por la vista), actualiza el modelo según sea necesario, y efectúa la comunicación con el servidor. El controlador también maneja los mensajes recibidos del servidor, actualizando el modelo y notificando a la vista.

2.3. Comunicación vía Sockets TCP

El servidor utiliza `TcpListener` para escuchar conexiones entrantes en un puerto dado. Cuando un cliente se conecta, se obtiene un `TcpStream` que representa la conexión bidireccional, la cual se clona en dos, para tener canales de lectura y escritura.

En el cliente se establece una conexión `TcpStream` al servidor especificando la dirección IP y puerto. Esta conexión se mantiene abierta durante toda la sesión del usuario.

Toda la lógica de la comunicación vía sockets se abstrajo en estructuras `NetworkReader` y `NetworkWriter` que envuelven los streams de red, proporcionando métodos para enviar y recibir mensajes de forma segura. Estos se comunican utilizando JSON como formato de serialización. Se usa un `mcsp` (multi producer single consumer) para tener una forma de comunicación entre los hilos que atienden las entradas de los clientes, y los hilos que escriben o envían mensajes a los clientes.

3. Protocolo de Comunicación

Se implementó el protocolo de comunicación especificado, donde los posibles mensajes y respuestas se abstrajeron en enumeraciones, cada una con tipos de estructuras que reciben como parámetro lo que diferencia un mensaje de otro (e.g. todos los mensajes `Identify` son iguales, salvo el nombre del usuario a identificarse).

3.1. Formato JSON

Todos los mensajes se serializan en formato JSON utilizando la librería `serde_json`. Cada mensaje tiene un campo `type` que identifica su tipo, seguido de campos específicos según el tipo.

Por ejemplo, un mensaje IDENTIFY contiene `type` e `username`. Un mensaje de respuesta contiene `type`, `operation`, `result` e `extra`.

La serialización se realiza automáticamente mediante traits de `serde`, permitiendo que las enumeraciones definidas se conviertan a JSON y viceversa de forma transparente y en automático.

4. Implementación del Servidor

4.1. Estructura del Proyecto

El servidor está organizado en módulos:

main.rs: Punto de entrada que inicializa el servidor y comienza a escuchar conexiones.

server.rs: Contiene la lógica principal del servidor, incluyendo la estructura `Server` que gestiona el ciclo de vida de las conexiones.

handlers.rs: Implementa `ClientHandler`, una estructura que maneja la comunicación con un cliente con el servidor. Cada conexión obtiene su propio `ClientHandler` ejecutándose en un hilo separado, y este, a su vez, tiene una instancia del `mcsp` para avisarle al servidor que debe enviar mensajes.

models: Módulo que contiene estructuras de datos compartidas como `User`, `Room` y `ServerState`.

protocol: Módulo que define los tipos de mensajes y operaciones como enumeraciones. Este módulo es compartido por el cliente y el servidor.

4.2. Modelo

El modelo del servidor incluye:

User: Representa un usuario conectado con su nombre de usuario, estado actual (ACTIVE, AWAY, BUSY), y ID de conexión único en el servidor.

Room: Representa un cuarto de chat. Tiene un nombre, lista de miembros y una de lista de usuarios invitados pero no unidos.

ServerState: Estructura que mantiene el estado global del servidor, incluyendo diccionario de usuarios activos, diccionario de salas, etc.

El estado del servidor está protegido por `Arc Mutex ServerState`, permitiendo acceso seguro desde múltiples hilos de `ClientHandler`. El Arc es para que el estado tenga múltiples dueños (que no es por defecto en Rust) y el candado Mutex para mantener el estado sincronizado.

4.3. Manejo de Conexiones

El servidor utiliza `TcpListener::bind()` para escuchar en un puerto específico.

Para cada conexión aceptada, se crea un nuevo `ClientHandler` y se generan dos hilos dedicado

mediante `thread::spawn()`. Estos hilos manejan toda la comunicación con ese cliente específico durante su sesión.

El `ClientHandler` mantiene referencias al estado compartido del servidor y al sistema de broadcast, permitiendo actualizar el estado global y notificar a otros clientes.

4.4. Sistema de Broadcast

El servidor implementa un sistema de broadcast para distribuir mensajes a múltiples clientes. Se utiliza una estructura `Broadcaster` que mantiene un diccionario de canales `mpsc::Sender`, uno por cada cliente conectado.

Cuando se necesita enviar un mensaje a múltiples clientes el broadcaster itera sobre los canales y envía el mensaje a cada uno. Los clientes que reciben estos mensajes tienen hilos dedicados escuchando en sus canales correspondientes.

Este broadcaster en el fondo, usa los módulos de `NetworkConnection` para la comunicación con sockets. Su principal objetivo, es tener los canales de comunicación y que cada hilo pueda avisarle al hilo principal qué mensaje debe enviar.

4.5. Procesamiento de Mensajes

El `ClientHandler` implementa un método `handle_message()` que recibe un mensaje del cliente y lo procesa según su tipo de enumeración.

Para cada tipo de mensaje, existe un manejador específico (e.g., `handle_identify()`, `handle_text()`, `handle_new_room()`).

4.6. Gestión de Errores

El servidor maneja errores de varias categorías:

Errores de red: Si la lectura del socket falla, se asume que el cliente se desconectó. El handler limpia el estado (remueve usuario, abandona salas) y termina.

Errores de lógica: Si una operación falla (usuario no existe, sala no existe, usuario no invitado), se envía un mensaje del tipo `RESPONSE` con el código de error específico, según lo especificado en el protocolo.

Errores de serialización: Si un mensaje JSON es inválido, el servidor lo ignora.

5. Implementación del Cliente

5.1. Estructura del Proyecto

El cliente está organizado en varios módulos

main.rs: Punto de entrada que inicializa la aplicación, crea el controlador y comienza los loops de interacción tanto con la entrada del usuario como con el servidor.

controller/chat_controller.rs: Implementa `ChatController` que orquesta la comunicación entre vista, modelo y servidor, según el patrón MVC.

model: Contiene estructuras de datos locales como `ChatModel`, `RemoteUser`, `RemoteRoom`, que son abstracciones de parte del modelo del servidor, pero solo aquello que corresponde al chat.

view/console_view.rs: Implementa el trait `ConsoleView` que proporciona una interfaz de usuario en línea de comandos.

network: Módulo con abstracciones de red `NetworkReader` y `NetworkWriter`, análogas a las del servidor.

5.2. Modelo

El modelo del cliente mantiene:

LocalUser: Información del usuario actual (nombre de usuario, estado de conexión). El usuario local es el que físicamente usa la aplicación de chat.

RemoteUser: Información de otros usuarios conectados (nombre, estado actual) al chat.

RemoteRoom: Información de los cuartos (nombre, miembros) del chat.

ChatModel: Estructura principal que contiene diccionarios de usuarios remotos, salas, estado de conexión, y lista de mensajes recibidos.

5.3. Vista

La `ConsoleView` implementa el trait `ChatView`, proporcionando los métodos necesarios para actualizar la interfaz de usuario. Solo se usan impresiones a la salida estándar.

5.4. Controlador

El `ChatController` coordina el flujo entre vista y modelo. En particular, se encarga de:

Conexiones: Solicita información del servidor al usuario y establece conexión TCP.

Identificación: Pide nombre de usuario, envía el mensaje `IDENTIFY` al servidor.

Procesamiento de eventos: Captura comandos del usuario, los traduce a mensajes de protocolo que envía al servidor.

Manejo de respuestas: Procesa mensajes del servidor y actualiza el modelo y la vista.

5.5. Comunicación de Red

El cliente utiliza tres hilos para comunicación:

Hilo de lectura: Ejecuta `receive_messages_loop()`, continuamente recibe mensajes del servidor y los procesa mediante el controlador.

Hilo de escritura: Ejecuta `send_messages_loop()`, escucha en un canal `mpsc::Receiver` y envía mensajes al servidor.

Hilo principal: Ejecuta `user_interaction_loop()`, captura entrada del usuario y envía comandos por el canal al hilo de escritura.

5.6. Punto de Entrada

La función `main()` realiza:

- Crea instancias de modelo, vista y controlador
- Solicita información de servidor y nombre de usuario
- Establece conexión
- Genera tres hilos (lectura, escritura, interacción)
- Espera a que terminen

La lógica de la inicialización se abstrae en una estructura `App`, tratando de imitar Swing de Java.

6. Serialización/Deserialización

Las estructuras de datos del protocolo derivan las implementaciones de traits `Serialize` y `Deserialize` de `serde`. Esto permite convertir automáticamente entre enumeraciones y JSON usando `serde_json::to_string()` y `serde_json::from_str()`.

El `NetworkReader` utiliza deserialización para convertir JSON recibido a enumeraciones del protocolo. El `NetworkWriter` utiliza serialización para convertir mensajes a JSON antes de enviar.

Por lo tanto, en ninguna parte del código es necesario escribir cadenas JSON a mano.

7. Concurrencia

7.1. Uso de Hilos

7.2. Gestión de Estado Compartido

Servidor: El estado global del servidor (`ServerState`) se protege con `Arc Mutex ServerState`. `Arc` permite múltiples propietarios (múltiples hilos de cliente, pues cada hilo toma propiedad en Rust), y `Mutex` garantiza acceso exclusivo (sincronización).

Cuando un cliente necesita actualizar el estado (agregar usuario, crear sala), adquiere el lock del mutex, realiza la actualización, y lo libera. Esto nos asegura que solo un hilo a la vez puede modificar el estado del servidor.

Cliente: El controlador se protege con `Arc Mutex ChatController`, permitiendo que los tres hilos accedan y actualicen el estado del cliente de forma sincronizada.

7.3. Canales de Comunicación

El sistema utiliza canales `mpsc` (multi-productor, unico-consumidor) para comunicación entre hilos.

Servidor: El broadcaster mantiene un `Sender` por cliente. Cuando se necesita enviar mensaje a múltiples clientes, se envía por cada canal. Estos canales son los que le avisan al módulo de NetworkConnection que debe escribir en el socket.

Cliente: Un canal `mpsc` comunica el hilo de interacción con el hilo de escritura de red. El usuario ingresa comandos que se envían por el canal, y el hilo de escritura los recibe y transmite al servidor.

8. Pruebas

8.1. Pruebas Unitarias

Se incluyen pruebas para validadores y lógica de parseo.

Las pruebas validan:

Validación de nombres: Nombres vacíos, demasiado largos, caracteres especiales.

Validación de mensajes: Mensajes vacíos, límites de longitud.

9. Diagramas

9.1. De la arquitectura cliente-servidor

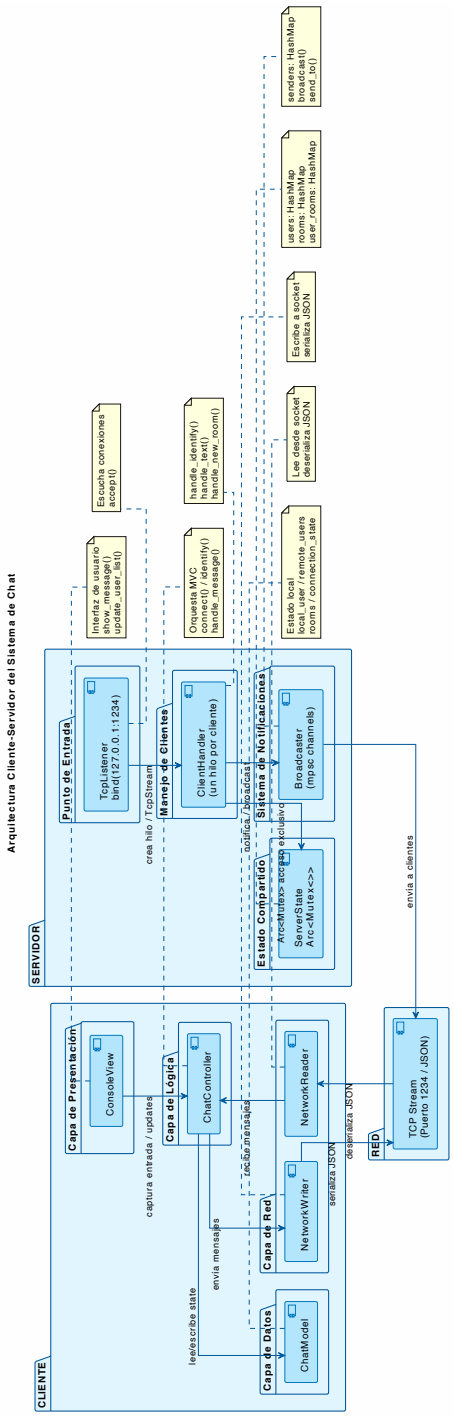
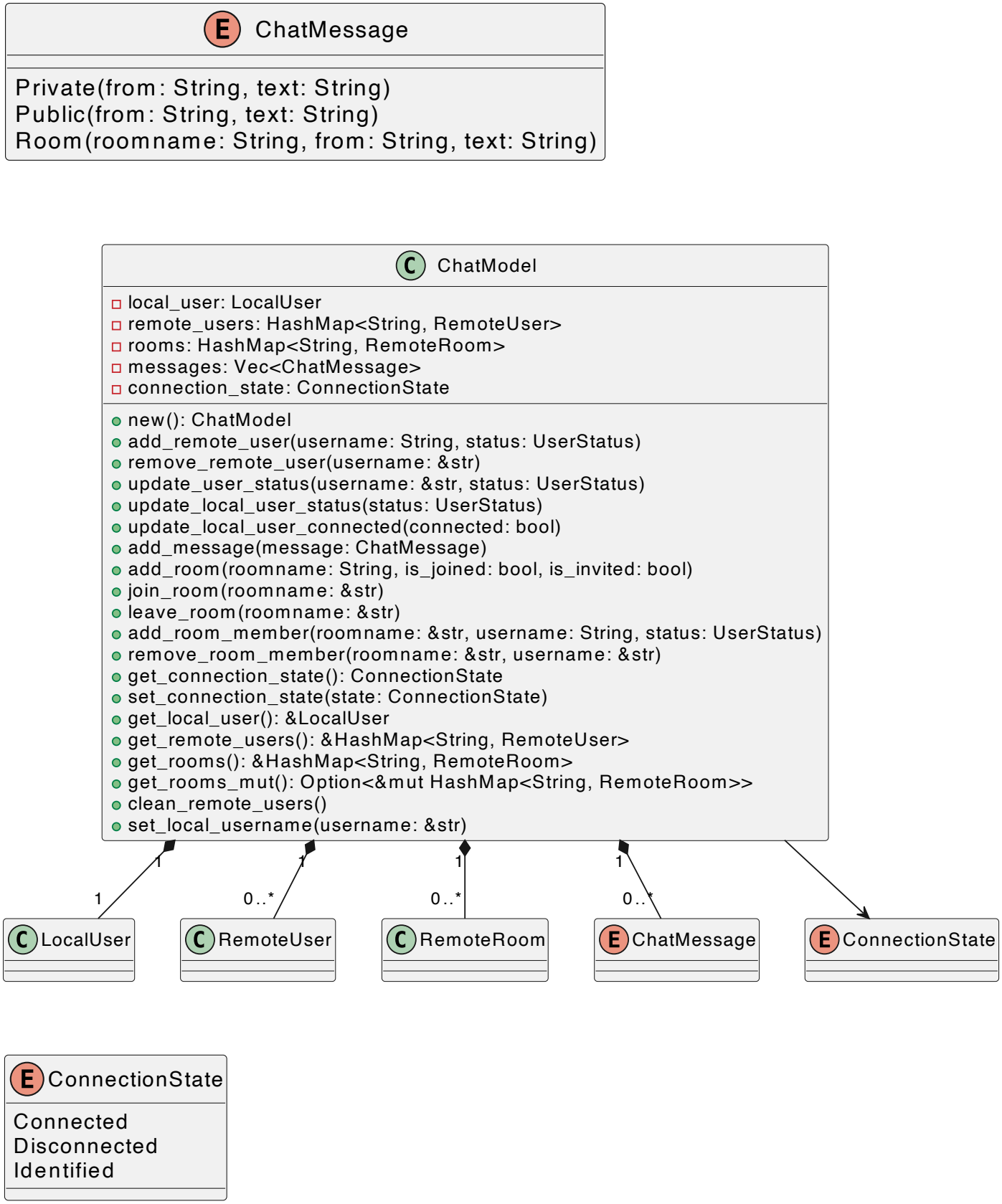
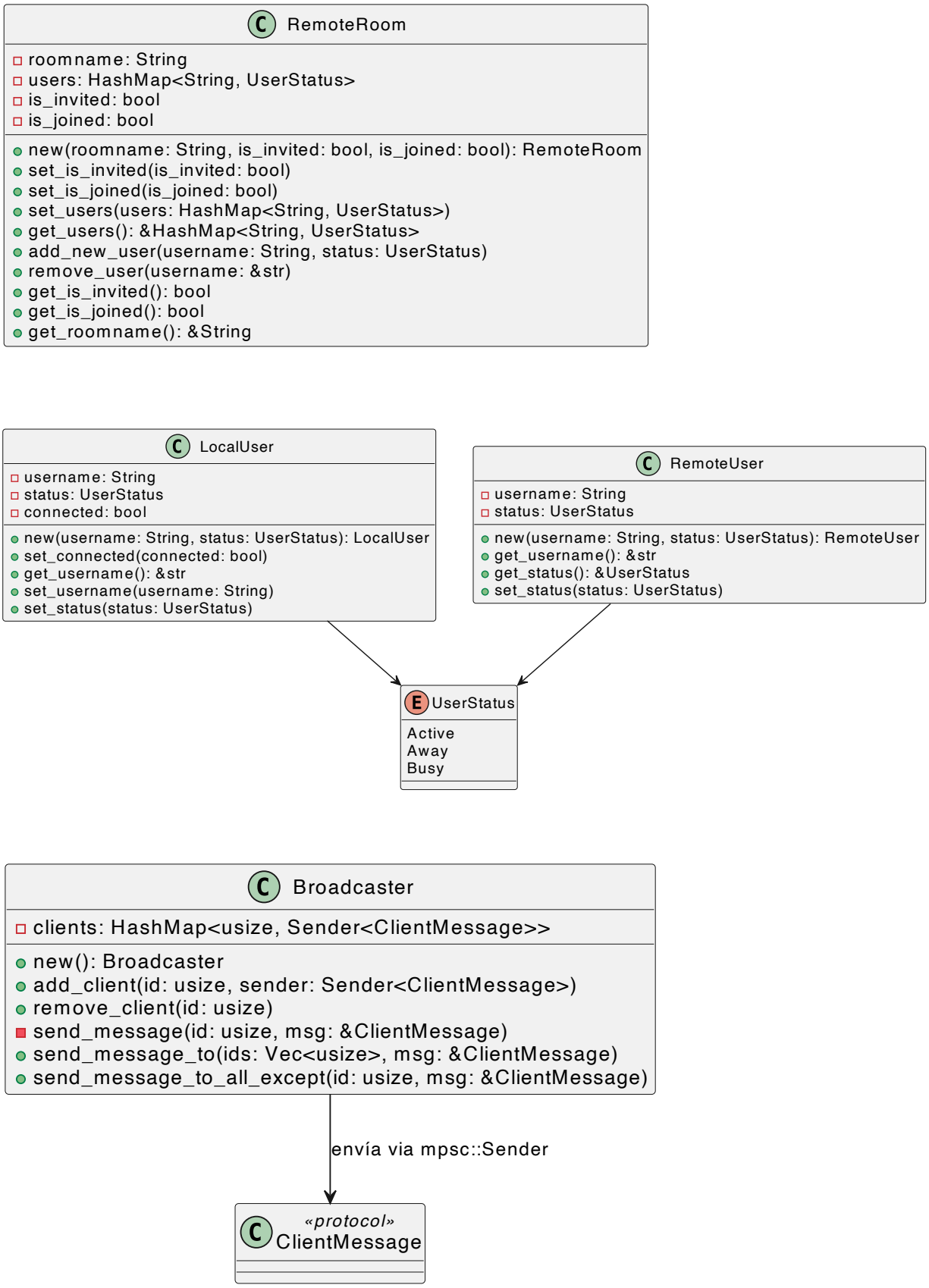
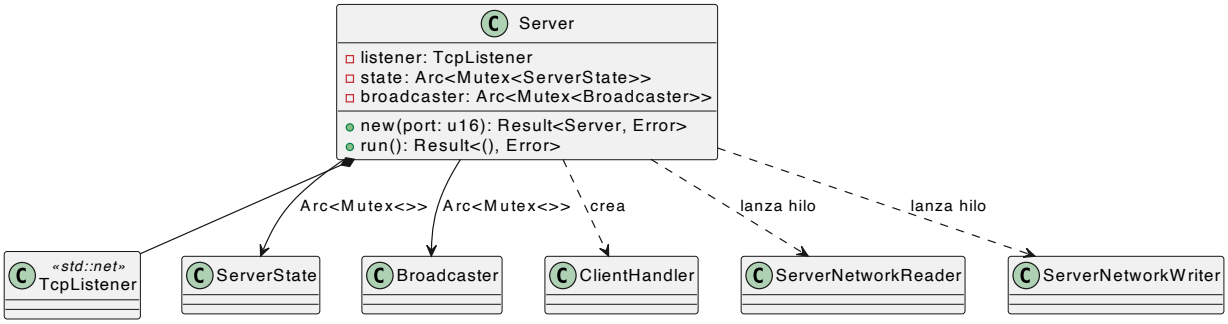
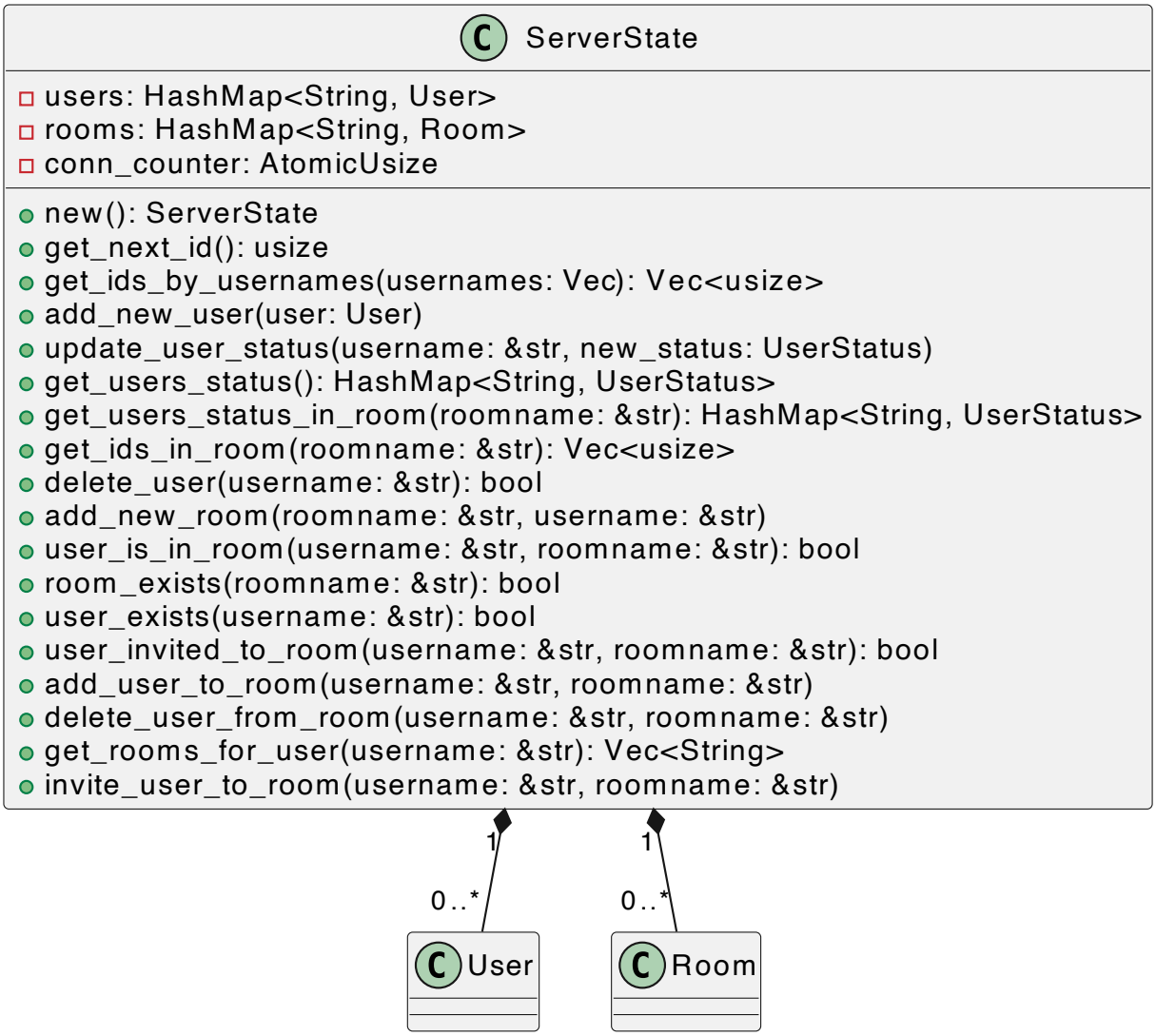


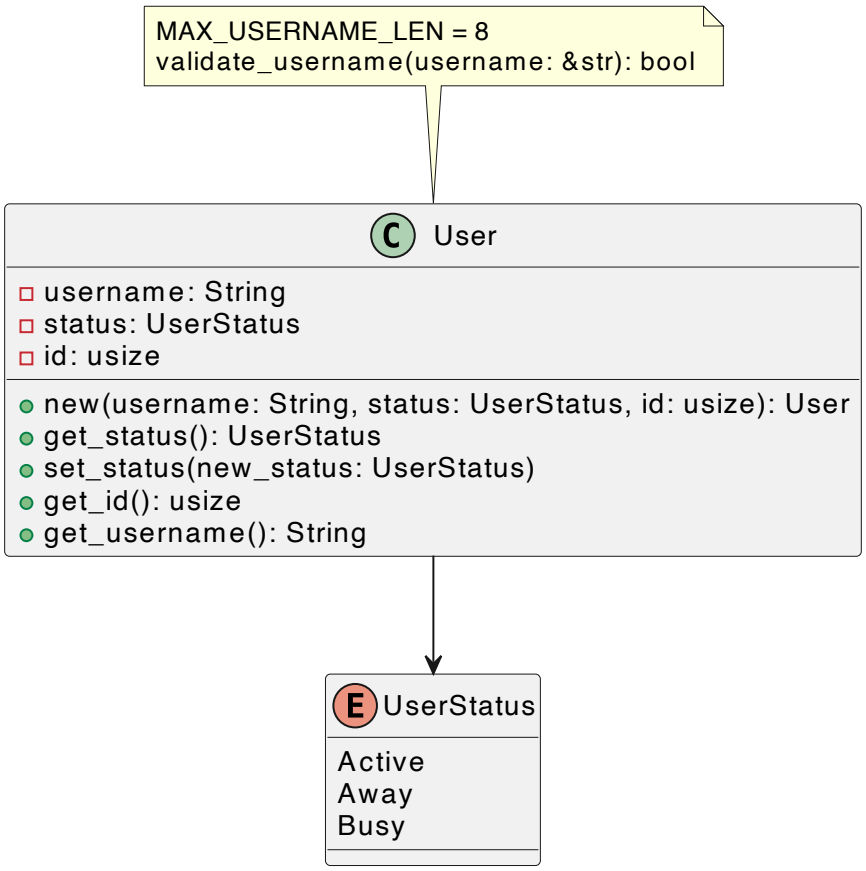
Figura 1: Arquitectura cliente-servidor del chat


ChatMessage
Private(from: String, text: String)
Public(from: String, text: String)
Room(roomname: String, from: String, text: String)
ChatModel
local_user: LocalUser
remote_users: HashMap<String, RemoteUser>
rooms: HashMap<String, RemoteRoom>
messages: Vec<ChatMessage>
connection_state: ConnectionState
new(): ChatModel
add_remote_user(username: String, status: UserStatus)
remove_remote_user(username: &str)
update_user_status(username: &str, status: UserStatus)
update_local_user_status(status: UserStatus)
update_local_user_connected(connected: bool)
add_message(message: ChatMessage)
add_room(roomname: String, is_joined: bool, is_invited: bool)
join_room(roomname: &str)
leave_room(roomname: &str)
add_room_member(roomname: &str, username: String, status: UserStatus)
remove_room_member(roomname: &str, username: &str)
get_connection_state(): ConnectionState
set_connection_state(state: ConnectionState)
get_local_user(): &LocalUser
get_remote_users(): &HashMap<String, RemoteUser>
get_rooms(): &HashMap<String, RemoteRoom>
get_rooms_mut(): Option<&mut HashMap<String, RemoteRoom>>
clean_remote_users()
set_local_username(username: &str)
1
1
1
1
1
0..*
0..*
0..*
LocalUser
RemoteUser
RemoteRoom
ChatMessage
ConnectionState
ConnectionState
Connected
Disconnected
Identified


RemoteRoom
roomname: String
users: HashMap<String, UserStatus>
is_invited: bool
is_joined: bool
new(roomname: String, is_invited: bool, is_joined: bool): RemoteRoom
set_is_invited(is_invited: bool)
set_is_joined(is_joined: bool)
set_users(users: HashMap<String, UserStatus>)
get_users(): &HashMap<String, UserStatus>
add_new_user(username: String, status: UserStatus)
remove_user(username: &str)
get_is_invited(): bool
get_is_joined(): bool
get_roomname(): &String
LocalUser
username: String
status: UserStatus
connected: bool
new(username: String, status: UserStatus): LocalUser
set_connected(connected: bool)
get_username(): &str
set_username(username: String)
set_status(status: UserStatus)
RemoteUser
username: String
status: UserStatus
new(username: String, status: UserStatus): RemoteUser
get_username(): &str
get_status(): &UserStatus
set_status(status: UserStatus)
UserStatus
Active
Away
Busy
Broadcaster
clients: HashMap<usize, Sender<ClientMessage>>
new(): Broadcaster
add_client(id: usize, sender: Sender<ClientMessage>)
remove_client(id: usize)
send_message(id: usize, msg: &ClientMessage)
send_message_to(ids: Vec<usize>, msg: &ClientMessage)
send_message_to_all_except(id: usize, msg: &ClientMessage)
envía via mpsc::Sender
«protocol»
ClientMessage


Server
listener: TcpListener
state: Arc<Mutex<ServerState>>
broadcaster: Arc<Mutex<Broadcaster>>
new(port: u16): Result<Server, Error>
run(): Result<(), Error>
Arc<Mutex<>>
Arc<Mutex<>>
crea
lanza hilo
lanza hilo
«std::net»
TcpListener
ServerState
Broadcaster
ClientHandler
ServerNetworkReader
ServerNetworkWriter


ServerState
users: HashMap<String, User>
rooms: HashMap<String, Room>
conn_counter: AtomicUsize
new(): ServerState
get_next_id(): usize
get_ids_by_usernames(usernames: Vec): Vec<usize>
add_new_user(user: User)
update_user_status(username: &str, new_status: UserStatus)
get_users_status(): HashMap<String, UserStatus>
get_users_status_in_room(roomname: &str): HashMap<String, UserStatus>
get_ids_in_room(roomname: &str): Vec<usize>
delete_user(username: &str): bool
add_new_room(roomname: &str, username: &str)
user_is_in_room(username: &str, roomname: &str): bool
room_exists(roomname: &str): bool
user_exists(username: &str): bool
user_invited_to_room(username: &str, roomname: &str): bool
add_user_to_room(username: &str, roomname: &str)
delete_user_from_room(username: &str, roomname: &str)
get_rooms_for_user(username: &str): Vec<String>
invite_user_to_room(username: &str, roomname: &str)
1
0..*
1
0..*
User
Room


MAX_USERNAME_LEN = 8
validate_username(username: &str): bool
User
username: String
status: UserStatus
id: usize
new(username: String, status: UserStatus, id: usize): User
get_status(): UserStatus
set_status(new_status: UserStatus)
get_id(): usize
get_username(): String
UserStatus
Active
Away
Busy